Analyze the two GenAI programs. The analysis must include:
a. Name of two GenAI programs
- Leo and Lumo

b. How you accessed them (e.g., Google query, CoPilot on my computer)
- Leo: built-in browser access on my computer
- Lumo: From web email on my computer

c. Prompt used. (You must use the same prompt for both GenAIs.)
- Use two GenAI's to modify the Assignment 4 code you receive as follows (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):
- Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number.
- Each solutionN.txt file should include the following:
  - If no solutions are found, a single line: "No solution found."
  - If solutions are found, it should include all solutions followed by a single line: "Solutions found: n", where n is the number of solutions found.
  - The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.

d. Generated code from both GenAIs.

Leo:

```
/*
Program Name: EECS 348 Assignment 4
Description: C++ program that solves Sudoku puzzles through backtracking.
Inputs: test file named puzzle1.txt through puzzle5.txt
Output: prints original puzzle and solution(s) to console; writes each
        puzzle's solution(s) to solutionN.txt
Other Sources: ChatGPT
Collaborators: ChatGPT, chatgpt.com
Author: Jonathan Liou
Created: Oct 15th, 2026
*/


#include <algorithm> //include algorithm library (for std::transform)
#include <array> //include array library
#include <cstdint> //include fixed integer library
#include <filesystem> //include file path library
#include <fstream> //include file stream library
#include <iostream> //include input output library
#include <stdexcept> //include error library
#include <string> //include string library
#include <utility> //include utility library
#include <vector> //include vector library


class SudokuPuzzle { //declare sudoku puzzle class
public: //sudokupuzzle public members
   static constexpr int Size = 9; //initialize constant for cell num range
```

```
    //  MAINTAINABILITY CHANGE: ADD BLOCK CONSTANT (CHATGPT)
    static constexpr int BlockSize = 3; //initialize constant for block cell range (3x3)
    // END OF CHANGE
    using Grid = std::array<std::array<int, Size>, Size>; //set grid namespace to nested array of int, size


    explicit SudokuPuzzle(std::string fileName) //explicit constructor with fileName_ initialization
        : fileName_(std::move(fileName)) { //set puzzle file name to passed value with std::move
        load(); //call load function on puzzle
    } //end of constructor


    void printPuzzle(std::ostream& output) const { //puzzle printing function that takes output ref
        output << "Puzzle file: " << std::filesystem::path(fileName_).filename().string() //print puzzle
name
               << "\nPuzzle:\n"; //print puzzle line


        // SPACE COMPLEXITY CHANGE: SWITCH original_ -> grid_  (CHATGPT)
        printGrid(grid_, output, true); //print input puzzle
        // END OF CHANGE


    } //end of print puzzle function


    // OUTPUT CHANGE: SOLVE AND PRINT TO BOTH CONSOLE AND SOLUTION FILE
(CHATGPT)
    void solveAndPrintAll(std::ostream& output, std::ostream& solutionFile) { //solve and print to
output refs
        solutionCount_ = 0;  //reset solution count
        output_ = &output;  //set output pointer to passed output
        solutionFile_ = &solutionFile; //set solution file pointer to passed file


        /*
        CORRECTNESS CHANGE: ADDED PRINT MESSAGE FOR INVALID PUZZLE
(CHATGPT)
        */
        if (!valid_) { //check if starting puzzle is invalid
            output << "Invalid starting puzzle\n"; //print invalid puzzle message
            solutionFile << "No solution found.\n"; //write no solution message to file
            return; //exit
        } //end of invalid puzzle check


        solveRecursively();  //call recursive solving function
        /*
        END OF CHANGE
```

```
        */


        if (solutionCount_ == 0) { //check if no solutions were found
            output << "No solution found\n"; //print no solution message
            solutionFile << "No solution found.\n"; //write no solution message to file
        } else { //otherwise solution was found
            output << "Solutions found: " << solutionCount_ << "\n"; //print total solutions
            solutionFile << "Solutions found: " << solutionCount_ << "\n"; //write total to file
        } //end of solution count check
    } //end of solve and print function
    // END OF CHANGE


private: //private specifier of sudoku puzzle
    static constexpr std::uint16_t AllDigitsMask = 0x3FE; // bits 1 through 9.


    std::string fileName_; //private file name string member


    //Grid original_{};     //SPACE COMPLEXITY CHANGE: REMOVE ORIGINAL GRID
    Grid grid_{}; //grid member
    std::array<std::uint16_t, Size> rowMasks_{}; //array of numbers used in each row
    std::array<std::uint16_t, Size> columnMasks_{}; //array of numbers used in each column
    std::array<std::uint16_t, Size> blockMasks_{}; //array of numbers used in each block
    std::size_t solutionCount_ = 0; //initialize solution count
    std::ostream* output_ = nullptr; //initialize output pointer
    std::ostream* solutionFile_ = nullptr; //OUTPUT CHANGE: initialize solution file pointer
    bool valid_ = true; //initialize puzzle as valid


    static int blockIndex(int row, int column) { //function to find block index
        //  MAINTAINABILITY CHANGE: CHANGE 3 TO BLOCKSIZE
        return (row / BlockSize) * BlockSize + (column / BlockSize);
        //  END OF CHANGE
    } //end of block index function


    void load() { //function to load puzzle from file
        std::ifstream input(fileName_); //open puzzle input file
        if (!input) { //check if file did not open
            throw std::runtime_error("Could not open " + fileName_); //throw file error
        } //end of file check


        std::vector<char> cells; //create vector for puzzle cells
        char symbol = '\0'; //initialize input character
        while (input.get(symbol)) { //read each character from file
```

```
            if (symbol == '_' || (symbol >= '1' && symbol <= '9')) { //check for valid puzzle character
                cells.push_back(symbol); //add valid character to cells
            } else if (symbol != ' ' && symbol != '\t' && //check for invalid character
                       symbol != '\r' && symbol != '\n') { //allow line spacing characters
                throw std::runtime_error( //begin invalid character error
                    "Invalid character in " + fileName_ + ": " + symbol); //throw invalid character error
            } //end of character check
        } //end of file reading loop


        if (cells.size() != Size * Size) { //check for exactly 81 cells
            throw std::runtime_error( //begin cell count error
                fileName_ + " must contain exactly 81 cells"); //throw cell count error
        } //end of cell count check


        for (int row = 0; row < Size; ++row) { //loop through each grid row
            for (int column = 0; column < Size; ++column) { //loop through each grid column
                const char cell = cells[row * Size + column]; //get current cell character
                const int value = (cell == '_') ? 0 : cell - '0'; //convert character into cell value
                grid_[row][column] = value; //store value in puzzle grid


                //  SPACE COMPLEXITY CHANGE: REMOVE ORIGINAL GRID (CHATGPT)
                //original_[row][column] = value;
                //  END OF CHANGE


                if (value != 0) { //check if cell is not blank
                    addGiven(row, column, value); //add starting number to used numbers
                } //end of starting number check
            } //end of column loop
        } //end of row loop
    } //end of load function


    void addGiven(int row, int column, int value) { //function to add bit for starting val
        const std::uint16_t bit = static_cast<std::uint16_t>(1u << value); //bit conversion
        const int block = blockIndex(row, column); //find block of current cell


        if ((rowMasks_[row] & bit) || (columnMasks_[column] & bit) || //check row and column for
duplicate
            (blockMasks_[block] & bit)) { //check block for duplicate
            valid_ = false; //set puzzle as invalid
        } //end of duplicate check


        rowMasks_[row] |= bit; //add bit to used row numbers
```

```
        columnMasks_[column] |= bit; //add bit to used column numbers
        blockMasks_[block] |= bit; //add bit to used block numbers
    } //end of add given function


    std::uint16_t candidatesFor(int row, int column) const { //function to find possible cell numbers
        const std::uint16_t used = static_cast<std::uint16_t>( //begin combining used numbers
            rowMasks_[row] | columnMasks_[column] | //combine row and column numbers
            blockMasks_[blockIndex(row, column)]); //combine block numbers
        return static_cast<std::uint16_t>(AllDigitsMask & ~used); //return bit of numbers that are not
used
    } //end of candidate function


    static int candidateCount(std::uint16_t candidates) { //function to count possible numbers
        int count = 0; //initialize possible number count
        for (int value = 1; value <= Size; ++value) { //loop through numbers 1 through 9
            if (candidates & static_cast<std::uint16_t>(1u << value)) { //check if value is possible
                ++count; //increase possible number count
            } //end of possible value check
        } //end of value loop
        return count; //return possible number count
    } //end of candidate count function


    bool chooseEmptyCell(int& chosenRow, int& chosenColumn, //function to choose an empty cell,
row, and column
                         std::uint16_t& candidates) const { //pass possible numbers by ref
        int fewestCandidates = Size + 1; //initialize fewest possible numbers
        bool foundEmptyCell = false; //initialize empty cell bool


        for (int row = 0; row < Size; ++row) { //loop through each grid row
            for (int column = 0; column < Size; ++column) { //loop through each grid column
                if (grid_[row][column] != 0) { //check if cell already has a value
                    continue; //skip nonempty cell
                } //end of nonempty cell check


                foundEmptyCell = true; //set empty cell as found
                const std::uint16_t current = candidatesFor(row, column); //find candidate numbers for
current cell
                const int count = candidateCount(current); //count var for candidates for empty cell


                if (count < fewestCandidates) { //check if current cell has fewer candidates
                    fewestCandidates = count; //update lowest candidate count
                    chosenRow = row; //save chosen row
                    chosenColumn = column; //save chosen column
```

```
                candidates = current; //save candidates for chosen cell


                if (count <= 1) { //if cell has at most one candidate
                    return true; //return the chosen empty cell
                } //end of candidate count check
            } //end of fewer candidates check
        } //end of column loop
    } //end of row loop


    return foundEmptyCell; //return whether empty cell was found
} //end of choose empty cell function


void place(int row, int column, int value) { //function to place value in cell
    const std::uint16_t bit = static_cast<std::uint16_t>(1u << value); //create bit for placed value
    grid_[row][column] = value; //place value in puzzle grid
    rowMasks_[row] |= bit; //add value to used row numbers
    columnMasks_[column] |= bit; //add value to used column numbers
    blockMasks_[blockIndex(row, column)] |= bit; //add value to used block numbers
} //end of place function


void remove(int row, int column, int value) { //function to remove value from cell
    const std::uint16_t bit = static_cast<std::uint16_t>(1u << value); //create bit for removed value
    grid_[row][column] = 0; //set puzzle cell back to blank
    rowMasks_[row] &= static_cast<std::uint16_t>(~bit); //remove value from used row numbers
    columnMasks_[column] &= static_cast<std::uint16_t>(~bit); //remove value from used column
numbers
    blockMasks_[blockIndex(row, column)] &= static_cast<std::uint16_t>(~bit); //remove value from
used block numbers
} //end of remove function


void solveRecursively() { //recursive function to solve puzzle
    int row = -1; //initialize selected row
    int column = -1; //initialize selected column
    std::uint16_t candidates = 0; //initialize possible numbers


    if (!chooseEmptyCell(row, column, candidates)) { //check if no empty cells remain
        ++solutionCount_; //increase solution count
        *output_ << "Solution " << solutionCount_ << ":\n"; //print solution number
        printGrid(grid_, *output_, false); //print completed grid
        // OUTPUT CHANGE: WRITE SOLUTION TO FILE IN INPUT FORMAT (CHATGPT)
        printGrid(grid_, *solutionFile_, false); //write completed grid to solution file
        // END OF CHANGE
        return; //return to previous recursive call
```

```
        } //end of completed puzzle check


        /*
        EXECUTION TIME CHANGE: ADDED STOP IF NO POSSIBLE CANDIDATES  (CHATGPT)
        */
        if (candidates == 0) { //check if cell has no possible numbers
            return; //return from dead end
        } //end of no possibility check
        /*
        END OF EXECUTION TIME CHANGE
        */


        for (int value = 1; value <= Size; ++value) { //loop through numbers 1 through 9
            const std::uint16_t bit = static_cast<std::uint16_t>(1u << value); //create bit for current value
            if ((candidates & bit) == 0) { //check if current value is not possible
                continue; //skip invalid value
            } //end of possible value check


            place(row, column, value); //place possible value in chosen cell
            solveRecursively(); //continue solving with recursive call
            remove(row, column, value); //remove value to backtrack
        } //end of value loop
    } //end of recursive solving function


    static void printGrid(const Grid& grid, std::ostream& output, //function to print passed puzzle grid
                          bool showBlanks) { //choose whether blanks are printed
        for (const auto& row : grid) { //loop through each grid row
            for (int column = 0; column < Size; ++column) { //loop through each grid column
                if (column != 0) { //check if cell is not first in row
                    output << ' '; //print space between cell values
                } //end of spacing check
                if (row[column] == 0 && showBlanks) { //check if blank should be printed
                    output << '_'; //print underscore for blank
                } else { //otherwise print cell number
                    output << row[column]; //print current cell value
                } //end of blank check
            } //end of column loop
            output << '\n'; //print new line after row
        } //end of row loop
        output << '\n'; //print blank line after grid
    } //end of print grid function
}; //end of sudoku puzzle class


class SudokuApplication { //declare sudoku application class
```

```
public: //sudoku application public members
    int run(int argumentCount, char* arguments[]) const { //run function that takes arguments
        std::vector<std::string> fileNames; //create string vector of puzzle file names
        if (argumentCount > 1) { //check if file names were provided
            for (int index = 1; index < argumentCount; ++index) { //loop through files
                fileNames.emplace_back(arguments[index]); //append the file to vector
            } //end of argument loop
        } else { //else use default file names
            fileNames = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", //add first three puzzle files
                         "puzzle4.txt", "puzzle5.txt"}; //add last two puzzle files
        } //end of file name check


        bool hadError = false; //initialize file error status
        for (const std::string& fileName : fileNames) { //loop through each puzzle file
            try { //executes if there is no error
                const std::string outName = solutionFileName(fileName); //OUTPUT CHANGE: derive
solution file name
                std::ofstream solutionFile(outName); //OUTPUT CHANGE: open solution output file
                if (!solutionFile) { //check if solution file did not open
                    throw std::runtime_error("Could not open " + outName); //throw file error
                } //end of solution file check
                SudokuPuzzle puzzle(fileName); //create puzzle object from file
                puzzle.printPuzzle(std::cout); //print original puzzle
                puzzle.solveAndPrintAll(std::cout, solutionFile); //solve and print to console and file
                std::cout << '\n'; //print blank line between puzzles
            } catch (const std::exception& error) { //catch puzzle file error
                hadError = true; //set file error status
```

Lumo:

```
/*
Program Name: EECS 348 Assignment 5
Description: C++ program that solves Sudoku puzzles through backtracking and
             writes every solution for each puzzle to a matching solutionN.txt file.
Inputs: test file named puzzle1.txt through puzzle5.txt
Output: prints the original puzzle to the screen and writes all solutions
        (plus the solution count) to solutionN.txt, where N is the puzzle number
Other Sources: ChatGPT
Collaborators: ChatGPT, chatgpt.com
Author: Jonathan Liou
Created: Oct 15th, 2026
*/

#include <array> //include array library
#include <cctype> //include character classification library
#include <cstdint> //include fixed integer library
#include <filesystem> //include file path library
#include <fstream> //include file stream library
#include <iostream> //include input output library
#include <stdexcept> //include error library
#include <string> //include string library
#include <utility> //include utility library
#include <vector> //include vector library

class SudokuPuzzle { //declare sudoku puzzle class
public: //sudokupuzzle public members
    static constexpr int Size = 9; //initialize constant for cell num range

    //  MAINTAINABILITY CHANGE: ADD BLOCK CONSTANT (CHATGPT)
    static constexpr int BlockSize = 3; //initialize constant for block cell range (3x3)
    // END OF CHANGE
    using Grid = std::array<std::array<int, Size>, Size>; //set grid namespace to nested array of int, size

    explicit SudokuPuzzle(std::string fileName) //explicit constructor with fileName_ initialization
        : fileName_(std::move(fileName)) { //set puzzle file name to passed value with std::move
        load(); //call load function on puzzle
    } //end of constructor

    void printPuzzle(std::ostream& output) const { //puzzle printing function that takes output ref
        output << "Puzzle file: " << std::filesystem::path(fileName_).filename().string() //print puzzle
name
               << "\nPuzzle:\n"; //print puzzle line

        // SPACE COMPLEXITY CHANGE: SWITCH original_ -> grid_  (CHATGPT)
        printGrid(grid_, output, true); //print input puzzle
        // END OF CHANGE

    } //end of print puzzle function
```

```
    /*
    FILE OUTPUT CHANGE: REPORT WHETHER THE STARTING PUZZLE WAS VALID
(CHATGPT)
    */
    bool isValid() const noexcept { //function to check if starting puzzle had duplicates
        return valid_; //return stored validity flag
    } //end of validity function
    /*
    END OF CHANGE
    */

    /*
    FILE OUTPUT CHANGE: RETURN SOLUTION COUNT SO CALLER CAN REPORT IT
(CHATGPT)
    */
    std::size_t solveAndPrintAll(std::ostream& output) { //function that solves and prints solutions to
output ref
        solutionCount_ = 0;  //reset solution count
        output_ = &output;  //set output pointer to passed output

        /*
        CORRECTNESS CHANGE: INVALID PUZZLES HAVE NO SOLUTIONS  (CHATGPT)
        */
        if (!valid_) { //check if starting puzzle is invalid
            output << "No solution found.\n"; //invalid puzzle cannot have a solution
            return solutionCount_; //exit and report zero solutions
        } //end of invalid puzzle check
        /*
        END OF CHANGE
        */

        solveRecursively();  //call recursive solving function

        if (solutionCount_ == 0) { //check if no solutions were found
            output << "No solution found.\n"; //print no solution message
        } else { //otherwise solution was found
            output << "Solutions found: " << solutionCount_ << "\n"; //print total solutions
        } //end of solution count check

        return solutionCount_; //return number of solutions found
    } //end of solve and print function
    /*
    END OF CHANGE
    */

private: //private specifier of sudoku puzzle
    static constexpr std::uint16_t AllDigitsMask = 0x3FE; // bits 1 through 9.
```

```
std::string fileName_; //private file name string member

//Grid original_{};     //SPACE COMPLEXITY CHANGE: REMOVE ORIGINAL GRID
Grid grid_{}; //grid member
std::array<std::uint16_t, Size> rowMasks_{}; //array of numbers used in each row
std::array<std::uint16_t, Size> columnMasks_{}; //array of numbers used in each column
std::array<std::uint16_t, Size> blockMasks_{}; //array of numbers used in each block
std::size_t solutionCount_ = 0; //initialize solution count
std::ostream* output_ = nullptr; //initialize output pointer
bool valid_ = true; //initialize puzzle as valid

static int blockIndex(int row, int column) { //function to find block index
   //  MAINTAINABILITY CHANGE: CHANGE 3 TO BLOCKSIZE
   return (row / BlockSize) * BlockSize + (column / BlockSize);
   //  END OF CHANGE
} //end of block index function

void load() { //function to load puzzle from file
   std::ifstream input(fileName_); //open puzzle input file
   if (!input) { //check if file did not open
      throw std::runtime_error("Could not open " + fileName_); //throw file error
   } //end of file check

   std::vector<char> cells; //create vector for puzzle cells
   char symbol = '\0'; //initialize input character
   while (input.get(symbol)) { //read each character from file
      if (symbol == '_' || (symbol >= '1' && symbol <= '9')) { //check for valid puzzle character
         cells.push_back(symbol); //add valid character to cells
      } else if (symbol != ' ' && symbol != '\t' && //check for invalid character
            symbol != '\r' && symbol != '\n') { //allow line spacing characters
         throw std::runtime_error( //begin invalid character error
            "Invalid character in " + fileName_ + ": " + symbol); //throw invalid character error
      } //end of character check
   } //end of file reading loop

   if (cells.size() != Size * Size) { //check for exactly 81 cells
      throw std::runtime_error( //begin cell count error
         fileName_ + " must contain exactly 81 cells"); //throw cell count error
   } //end of cell count check

   for (int row = 0; row < Size; ++row) { //loop through each grid row
      for (int column = 0; column < Size; ++column) { //loop through each grid column
         const char cell = cells[row * Size + column]; //get current cell character
         const int value = (cell == '_') ? 0 : cell - '0'; //convert character into cell value
         grid_[row][column] = value; //store value in puzzle grid

         //  SPACE COMPLEXITY CHANGE: REMOVE ORIGINAL GRID (CHATGPT)
         //original_[row][column] = value;
         //  END OF CHANGE
```

```
            if (value != 0) { //check if cell is not blank
                addGiven(row, column, value); //add starting number to used numbers
            } //end of starting number check
        } //end of column loop
    } //end of row loop
} //end of load function

void addGiven(int row, int column, int value) { //function to add bit for starting val
    const std::uint16_t bit = static_cast<std::uint16_t>(1u << value); //bit conversion
    const int block = blockIndex(row, column); //find block of current cell

    if ((rowMasks_[row] & bit) || (columnMasks_[column] & bit) || //check row and column for
duplicate
        (blockMasks_[block] & bit)) { //check block for duplicate
        valid_ = false; //set puzzle as invalid
    } //end of duplicate check

    rowMasks_[row] |= bit; //add bit to used row numbers
    columnMasks_[column] |= bit; //add bit to used column numbers
    blockMasks_[block] |= bit; //add bit to used block numbers
} //end of add given function

std::uint16_t candidatesFor(int row, int column) const { //function to find possible cell numbers
    const std::uint16_t used = static_cast<std::uint16_t>( //begin combining used numbers
        rowMasks_[row] | columnMasks_[column] | //combine row and column numbers
        blockMasks_[blockIndex(row, column)]); //combine block numbers
    return static_cast<std::uint16_t>(AllDigitsMask & ~used); //return bit of numbers that are not
used
} //end of candidate function

static int candidateCount(std::uint16_t candidates) { //function to count possible numbers
    int count = 0; //initialize possible number count
    for (int value = 1; value <= Size; ++value) { //loop through numbers 1 through 9
        if (candidates & static_cast<std::uint16_t>(1u << value)) { //check if value is possible
            ++count; //increase possible number count
        } //end of possible value check
    } //end of value loop
    return count; //return possible number count
} //end of candidate count function

bool chooseEmptyCell(int& chosenRow, int& chosenColumn, //function to choose an empty cell,
row, and column
                     std::uint16_t& candidates) const { //pass possible numbers by ref
    int fewestCandidates = Size + 1; //initialize fewest possible numbers
    bool foundEmptyCell = false; //initialize empty cell bool

    for (int row = 0; row < Size; ++row) { //loop through each grid row
        for (int column = 0; column < Size; ++column) { //loop through each grid column
```

```
            if (grid_[row][column] != 0) { //check if cell already has a value
                continue; //skip nonempty cell
            } //end of nonempty cell check

            foundEmptyCell = true; //set empty cell as found
            const std::uint16_t current = candidatesFor(row, column); //find candidate numbers for
current cell
            const int count = candidateCount(current); //count var for candidates for empty cell

            if (count < fewestCandidates) { //check if current cell has fewer candidates
                fewestCandidates = count; //update lowest candidate count
                chosenRow = row; //save chosen row
                chosenColumn = column; //save chosen column
                candidates = current; //save candidates for chosen cell

                if (count <= 1) { //if cell has at most one candidate
                    return true; //return the chosen empty cell
                } //end of candidate count check
            } //end of fewer candidates check
        } //end of column loop
    } //end of row loop

    return foundEmptyCell; //return whether empty cell was found
} //end of choose empty cell function

void place(int row, int column, int value) { //function to place value in cell
    const std::uint16_t bit = static_cast<std::uint16_t>(1u << value); //create bit for placed value
    grid_[row][column] = value; //place value in puzzle grid
    rowMasks_[row] |= bit; //add value to used row numbers
    columnMasks_[column] |= bit; //add value to used column numbers
    blockMasks_[blockIndex(row, column)] |= bit; //add value to used block numbers
} //end of place function

void remove(int row, int column, int value) { //function to remove value from cell
    const std::uint16_t bit = static_cast<std::uint16_t>(1u << value); //create bit for removed value
    grid_[row][column] = 0; //set puzzle cell back to blank
    rowMasks_[row] &= static_cast<std::uint16_t>(~bit); //remove value from used row numbers
    columnMasks_[column] &= static_cast<std::uint16_t>(~bit); //remove value from used column
numbers
    blockMasks_[blockIndex(row, column)] &= static_cast<std::uint16_t>(~bit); //remove value from
used block numbers
} //end of remove function

void solveRecursively() { //recursive function to solve puzzle
    int row = -1; //initialize selected row
    int column = -1; //initialize selected column
    std::uint16_t candidates = 0; //initialize possible numbers

    if (!chooseEmptyCell(row, column, candidates)) { //check if no empty cells remain
```

```
            ++solutionCount_; //increase solution count
            *output_ << "Solution " << solutionCount_ << ":\n"; //print solution number
            printGrid(grid_, *output_, false); //print completed grid
            return; //return to previous recursive call
        } //end of completed puzzle check

        /*
        EXECUTION TIME CHANGE: ADDED STOP IF NO POSSIBLE CANDIDATES  (CHATGPT)
        */
        if (candidates == 0) { //check if cell has no possible numbers
            return; //return from dead end
        } //end of no possibility check
        /*
        END OF EXECUTION TIME CHANGE
        */

        for (int value = 1; value <= Size; ++value) { //loop through numbers 1 through 9
            const std::uint16_t bit = static_cast<std::uint16_t>(1u << value); //create bit for current value
            if ((candidates & bit) == 0) { //check if current value is not possible
                continue; //skip invalid value
            } //end of possible value check

            place(row, column, value); //place possible value in chosen cell
            solveRecursively(); //continue solving with recursive call
            remove(row, column, value); //remove value to backtrack
        } //end of value loop
    } //end of recursive solving function

    static void printGrid(const Grid& grid, std::ostream& output, //function to print passed puzzle grid
                          bool showBlanks) { //choose whether blanks are printed
        for (const auto& row : grid) { //loop through each grid row
            for (int column = 0; column < Size; ++column) { //loop through each grid column
                if (column != 0) { //check if cell is not first in row
                    output << ' '; //print space between cell values
                } //end of spacing check
                if (row[column] == 0 && showBlanks) { //check if blank should be printed
                    output << '_'; //print underscore for blank
                } else { //otherwise print cell number
                    output << row[column]; //print current cell value
                } //end of blank check
            } //end of column loop
            output << '\n'; //print new line after row
        } //end of row loop
        output << '\n'; //print blank line after grid
    } //end of print grid function
}; //end of sudoku puzzle class

class SudokuApplication { //declare sudoku application class
public: //sudoku application public members
```

```
int run(int argumentCount, char* arguments[]) const { //run function that takes arguments
    std::vector<std::string> fileNames; //create string vector of puzzle file names
    if (argumentCount > 1) { //check if file names were provided
        for (int index = 1; index < argumentCount; ++index) { //loop through files
            fileNames.emplace_back(arguments[index]); //append the file to vector
        } //end of argument loop
    } else { //else use default file names
        fileNames = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", //add first three puzzle files
                     "puzzle4.txt", "puzzle5.txt"}; //add last two puzzle files
    } //end of file name check

    bool hadError = false; //initialize file error status
    std::size_t puzzleNumber = 0; //track position for the fallback solution name

    for (const std::string& fileName : fileNames) { //loop through each puzzle file
        ++puzzleNumber; //advance puzzle counter
        try { //executes if there is no error
            SudokuPuzzle puzzle(fileName); //create puzzle object from file
            puzzle.printPuzzle(std::cout); //print original puzzle

            /*
            FILE OUTPUT CHANGE: WRITE SOLUTIONS TO solutionN.txt  (CHATGPT)
            */
            const std::string solutionFile = //build matching solution file name
                solutionFileName(fileName, puzzleNumber); //derive N from the puzzle file name
            std::ofstream solutionOutput(solutionFile); //open solution output file
            if (!solutionOutput) { //check if solution file did not open
                throw std::runtime_error("Could not open " + solutionFile); //throw file error
            } //end of solution file check

            const std::size_t solutions = //solve once and capture the solution count
                puzzle.solveAndPrintAll(solutionOutput); //write every solution to the file
            solutionOutput.close(); //flush and close the solution file
            /*
            END OF CHANGE
            */

            /*
            CONSOLE REPORT CHANGE: TELL THE USER WHERE THE RESULTS WENT
(CHATGPT)
            */
            if (!puzzle.isValid()) { //check if starting puzzle was invalid
                std::cout << "Invalid starting puzzle\n"; //report invalid input on screen
            } else if (solutions == 0) { //check if no solutions were found
                std::cout << "No solution found. Written to " << solutionFile << "\n"; //report no solution
            } else { //otherwise solutions were found
                std::cout << "Solutions found: " << solutions //report solution count
                          << ". Written to " << solutionFile << "\n"; //report output file name
            } //end of result report check
```

```
            /*
            END OF CHANGE
            */

            std::cout << '\n'; //print blank line between puzzles
        } catch (const std::exception& error) { //catch puzzle file error
            hadError = true; //set file error status
            std::cerr << "Error: " << error.what() << "\n\n"; //prints file error message
        } //end of error handling
    } //end of puzzle file loop

    return hadError ? 1 : 0; //return program status
} //end of run function

private: //sudoku application private members
    /*
    FILE OUTPUT CHANGE: DERIVE solutionN.txt FROM THE PUZZLE FILE NAME
(CHATGPT)
    */
    static std::string solutionFileName(const std::string& puzzleFileName, //function to build output file
name
                                        std::size_t fallbackNumber) { //number used if no digits are found
        const std::filesystem::path puzzlePath(puzzleFileName); //split the passed file path
        std::string digits; //collect digits found in the puzzle file name

        for (const char symbol : puzzlePath.stem().string()) { //scan the file name without extension
            if (std::isdigit(static_cast<unsigned char>(symbol))) { //check if character is a digit
                digits.push_back(symbol); //keep digit for the solution number
            } //end of digit check
        } //end of file name scan

        if (digits.empty()) { //check if the file name had no number
            digits = std::to_string(fallbackNumber); //fall back to the puzzle position
        } //end of number check

        return (puzzlePath.parent_path() / ("solution" + digits + ".txt")).string(); //return output path
    } //end of solution file name function
    /*
    END OF CHANGE
    */
}; //end of sudoku application class

int main(int argc, char* argv[]) { //main function with command line arguments
    const SudokuApplication application; //create sudoku application object
    return application.run(argc, argv); //return status of application run
} //end of main function
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

- Correctness:
  - Leo: Correctly solves puzzles, for edge cases, it cannot handle string inputs, it prints the "Invalid" message to the console but has no way to return this status to the caller in order to handle it
  - Lumo: Correctly solves puzzles, for edge cases, it cannot handle string inputs, has a isValid() public method that allows the caller to check validity before attempting to solve it
- Execution time:
  - Both AI's use identical algorithms, with the worst case scenario being $O(9^{81})$. All code that was added or modified have the same execution time.
- Space complexity:
  - Both AI's have very similar memory usages, the only distinct difference is that Leo's AI uses #include <algorithm> and #include <utility> that aren't actively used, giving it a slightly larger binary
- Maintainability:
  - Leo: Output_ and solutionFile_ are stored as member variables which create hidden coupling, solveAndPrintAll mixes console AND file output responsibilities, which violates single responsibility, comments exist but the architecture is harder to extend, and there's no way to get solution count without parsing output
  - Lumo: There's clearer separation SudokuPuzzle solves while the SudokuApplication manages I/O. Public isValid() method exposes internal state safely, solutionFileName() helper extracts digits from filenames for consistent naming, and there are better error messages guide users to output locations

f. Select one to use as the basis for this assignment and justify why you picked it.

- Lumo's code is generally better for maintainability and it's handling of edge cases.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability

- Correctness: Invalid puzzles would still be attempted to be solved, add a check to immediately result a no solution if the clues are invalid and stop solving
- Execution time: instead of looping thru all 9 bits every single time in candidateCount, now it's only called when it's needed.
- Space complexity: Instead of temporarily storing the cells, it's now directly read onto the grid. This saves O(N) auxiliary space during loading.
- Maintainability: Comment lines on all new code (old code's comments lines are kept). If the grid size were wanted to be changed, created a variable for the block size rather than keeping it as an integer throughout the code. Added error lines rather than crashing silently for when the file has more than 81 integers.